100 Most Used Pandas Functions in Data Science



What's inside: the 100 Pandas functions data scientists reach for most, grouped into 10 task-based sections. Each section has a quick-reference table and a short, copy-ready code example. Every example assumes the setup below, where `df` is a DataFrame.

Setup used in all examples

```
import pandas as pd
import numpy as np

df = pd.read_csv("data.csv")
```

Import / Export

#	Function	What it does
1	`pd.read_csv()`	Load a CSV file
2	`pd.read_excel()`	Load an Excel file
3	`pd.read_json()`	Load a JSON file
4	`pd.read_sql()`	Load data from a SQL query
5	`pd.read_parquet()`	Load a Parquet file
6	`df.to_csv()`	Save to CSV
7	`df.to_excel()`	Save to Excel
8	`df.to_json()`	Save to JSON
9	`df.to_sql()`	Write a DataFrame to a database
10	`df.to_parquet()`	Save to Parquet

Load and save in one go

```
df = pd.read_csv("sales.csv")
df = pd.read_excel("sales.xlsx", sheet_name="Q1")
df = pd.read_json("sales.json")
df = pd.read_sql("SELECT * FROM orders", conn)
df = pd.read_parquet("sales.parquet")
df.to_csv("clean.csv", index=False)
df.to_excel("clean.xlsx", index=False)
df.to_json("clean.json", orient="records")
df.to_sql("orders", conn, if_exists="replace")
df.to_parquet("clean.parquet")
```

> ⚠️ **Warning:** Pass `index=False` to `to_csv()` and `to_excel()` unless you want the index saved as an extra, unnamed column.

Inspecting Data

#	Function	What it does
11	`head()`	First rows (5 by default)
12	`tail()`	Last rows
13	`info()`	Column dtypes and non-null counts
14	`describe()`	Summary statistics
15	`shape`	(rows, columns)
16	`columns`	Column names
17	`dtypes`	Data type of each column
18	`sample()`	Random rows
19	`nunique()`	Count of unique values
20	`value_counts()`	Frequency of each value
21	`unique()`	Array of unique values
22	`memory_usage()`	Memory used per column

Quick first look at a dataset

```
df.head()                    # first 5 rows
df.tail(3)                   # last 3 rows
df.info()                    # dtypes + non-null counts
df.describe()                # count, mean, std, min, max
df.shape                     # (rows, columns)
df.columns                   # column names
df.dtypes                    # type of each column
df.sample(5)                 # 5 random rows
df["city"].nunique()         # number of unique cities
df["city"].value_counts()    # frequency of each city
df["city"].unique()          # array of unique cities
df.memory_usage(deep=True)   # bytes per column
```

🎯 **Key insight:** Run `info()` first on any new dataset. One call shows dtypes and null counts, so you know what needs cleaning before you do anything else.

🔴 Missing Values

#	Function	What it does
23	`isnull()`	Flag missing values
24	`notnull()`	Flag non-missing values
25	`isna()`	Alias of isnull()
26	`dropna()`	Remove rows or columns with nulls
27	`fillna()`	Fill nulls with a value
28	`ffill()`	Forward fill from the previous value
29	`bfill()`	Backward fill from the next value
30	`interpolate()`	Estimate missing values
31	`replace()`	Replace specific values

Find, drop and fill missing values

```
df.isnull().sum()           # nulls per column
df.notnull().sum()          # non-nulls per column
df.isna().mean()            # share of nulls per column
df.dropna(subset=["age"])   # drop rows missing age
df["age"].fillna(0)         # fill nulls with 0
df.ffill()                  # carry last value forward
df.bfill()                  # pull next value backward
df["temp"].interpolate()    # estimate the gaps
df.replace("N/A", np.nan)   # standardize null markers
```

⚠️ **Warning:** `dropna()` with no arguments deletes every row that has even one null, which can wipe out most of your data. Check `df.isna().sum()` first, or use `subset=`.

Selection / Filtering

#	Function	What it does
32	`loc[]`	Label-based selection
33	`iloc[]`	Position-based selection
34	`at[]`	Fast single-value access
35	`query()`	Filter using a string expression
36	`filter()`	Select columns by name pattern
37	`isin()`	Match against a list of values
38	`between()`	Values within a range
39	`nlargest()`	Top N values
40	`nsmallest()`	Bottom N values
41	`select_dtypes()`	Pick columns by dtype

Select and filter rows and columns

```
df.loc[0, "name"]          # by label
df.iloc[0:5, 0:3]          # by position
df.at[0, "name"]           # fastest single value
df.query("age > 30 and city == 'Delhi'")
df.filter(like="date")     # columns containing "date"
df[df["city"].isin(["Delhi", "Noida"])]
df[df["age"].between(20, 30)]
df.nlargest(5, "salary")   # top 5 by salary
df.nsmallest(5, "salary")  # bottom 5 by salary
df.select_dtypes(include="number")
```

🎯 **Key insight:** Use `loc` for labels and `iloc` for positions. A boolean mask inside `loc`, like `df.loc[df["age"] > 30, "name"]`, filters rows and picks columns in one step.

Cleaning / Transforming

#	Function	What it does
42	`drop()`	Remove rows or columns
43	`rename()`	Rename labels
44	`astype()`	Change dtype
45	`drop_duplicates()`	Remove duplicate rows
46	`duplicated()`	Flag duplicate rows
47	`assign()`	Add new columns
48	`apply()`	Apply a function along an axis
49	`map()`	Element-wise mapping on a Series
50	`transform()`	Same-shaped output (great with groupby)
51	`clip()`	Cap values at limits
52	`round()`	Round numbers

Clean and transform columns

```
df.drop(columns=["temp"])
df.rename(columns={"dob": "birth_date"})
df["age"] = df["age"].astype(int)
df.drop_duplicates(subset=["id"])
df.duplicated().sum()
df.assign(total=df["qty"] * df["price"])
df.apply(lambda r: r["a"] + r["b"], axis=1)
df["grade"].map({"A": 1, "B": 2})
df.groupby("city")["sales"].transform("mean")
df["score"].clip(0, 100)
df["price"].round(2)
```

⚠️ **Warning:** `apply()` with `axis=1` loops row by row and gets slow on big data. Prefer vectorized operations such as `df["a"] + df["b"]` whenever you can.

Sorting / Indexing

#	Function	What it does
53	`sort_values()`	Sort by values
54	`sort_index()`	Sort by index
55	`set_index()`	Make a column the index
56	`reset_index()`	Restore the default index
57	`reindex()`	Conform to a new index

Sort and re-index

```
df.sort_values("salary", ascending=False)
df.sort_index()
df = df.set_index("id")
df = df.reset_index(drop=True)
df = df.reindex(range(10), fill_value=0)
```

Aggregation / Statistics

#	Function	What it does
58	`groupby()`	Split-apply-combine
59	`agg()`	Multiple aggregations at once
60	`sum()`	Total
61	`mean()`	Average
62	`median()`	Middle value
63	`mode()`	Most frequent value
64	`std()`	Standard deviation
65	`var()`	Variance
66	`min()`	Minimum
67	`max()`	Maximum
68	`count()`	Non-null count
69	`cumsum()`	Cumulative sum
70	`cumprod()`	Cumulative product
71	`pct_change()`	Percentage change
72	`diff()`	Difference from the previous row
73	`rank()`	Rank values
74	`corr()`	Correlation matrix
75	`cov()`	Covariance
76	`quantile()`	Percentiles
77	`idxmax()`	Index of the max value
78	`idxmin()`	Index of the min value

Group and summarize

```
df.groupby("city")["sales"].sum()
df.groupby("city").agg({"sales": ["sum", "mean"]})
df["sales"].sum()
df["sales"].mean()
df["sales"].median()
df["sales"].mode()
df["sales"].std()
df["sales"].var()
df["sales"].min()
df["sales"].max()
df["sales"].count()
```

Trends, ranks and relationships

```
df["sales"].cumsum()
df["growth"].cumprod()
df["sales"].pct_change()
df["sales"].diff()
df["sales"].rank(ascending=False)
df.corr(numeric_only=True)
df.cov(numeric_only=True)
df["sales"].quantile(0.9)
df["sales"].idxmax()
df["sales"].idxmin()
```



Key insight: `groupby()` + `agg()` is the most-used pattern in analysis: split by a category, compute stats for each group, and combine the results in a single line.

Combining / Reshaping

#	Function	What it does
79	`pd.merge()`	SQL-style joins
80	`pd.concat()`	Stack DataFrames
81	`join()`	Join on the index
82	`pivot()`	Reshape long to wide
83	`pivot_table()`	Pivot with aggregation
84	`melt()`	Reshape wide to long
85	`stack()`	Columns to rows
86	`unstack()`	Rows to columns
87	`pd.crosstab()`	Frequency cross-tab
88	`explode()`	List items to separate rows

Join, reshape and cross-tabulate

```
pd.merge(orders, customers, on="cust_id", how="left")
pd.concat([jan, feb], ignore_index=True)
orders.join(customers)
df.pivot(index="date", columns="city", values="sales")
df.pivot_table(index="city", values="sales", aggfunc="sum")
df.melt(id_vars="city", var_name="month", value_name="amt")
df.stack()
df.unstack()
pd.crosstab(df["city"], df["gender"])
df.explode("tags")
```

⚠️ **Warning:** A wrong key or `how=` type can silently duplicate or drop rows in a merge. Compare `len(df)` before and after, or pass `validate="one_to_one"`.

String Operations

#	Function	What it does
89	`str.lower() / str.upper()`	Change case
90	`str.strip()`	Trim whitespace
91	`str.contains()`	Pattern search
92	`str.replace()`	Replace text
93	`str.split()`	Split into parts

Clean text columns

```
df["name"].str.lower()
df["name"].str.upper()
df["name"].str.strip()
df["email"].str.contains("@gmail")
df["phone"].str.replace("-", "")
df["name"].str.split(" ")
```

Datetime / Window / Binning

#	Function	What it does
94	`pd.to_datetime()`	Convert to datetime
95	`dt accessor`	Extract date parts: dt.year, dt.month, dt.dayofweek
96	`resample()`	Time-based grouping
97	`rolling()`	Moving-window calculations
98	`shift()`	Lag or lead values
99	`pd.cut() / pd.qcut()`	Bin values into ranges or quantiles
100	`pd.get_dummies()`	One-hot encoding

Dates, windows and binning

```
df["date"] = pd.to_datetime(df["date"])
df["date"].dt.year
df["date"].dt.month
df["date"].dt.dayofweek
df.resample("MS", on="date")["sales"].sum()
df["sales"].rolling(7).mean()
df["sales"].shift(1)
pd.cut(df["age"], bins=[0, 18, 60, 100])
pd.qcut(df["salary"], q=4)
pd.get_dummies(df["city"])
```



Key insight: Convert date columns with `pd.to_datetime()` first. The `.dt` accessor, `resample()` and date-based rolling windows all depend on it.

🏁 At a glance

Section	#	Count	Go-to functions	Use it to
Import / Export	1-10	10	`read_csv, read_sql, to_csv`	Get data in and out
Inspecting Data	11-22	12	`head, info, describe, value_counts`	Understand a new dataset
Missing Values	23-31	9	`isnull, dropna, fillna`	Find and handle nulls
Selection / Filtering	32-41	10	`loc, iloc, query, isin`	Pick the rows and columns you need
Cleaning / Transforming	42-52	11	`drop, rename, astype, apply`	Fix types, names and duplicates
Sorting / Indexing	53-57	5	`sort_values, set_index`	Order rows and manage the index
Aggregation / Statistics	58-78	21	`groupby, agg, corr, rank`	Summarize and compare
Combining / Reshaping	79-88	10	`merge, concat, pivot_table, melt`	Join and reshape tables
String Operations	89-93	5	`str.contains, str.replace`	Clean text columns
Datetime / Window / Binning	94-100	7	`to_datetime, resample, rolling`	Time series, bins, encoding
Total	**1-100**	**100**		